OPEN SOURCE SOFTWARE DEVELOPMENT

Cybersecurity Incident Reporting and Tracking System

Phase 1: Project Proposal Document

Course Name	Open Source Software Development (OSSD)
Student Name	Areeba Abbas
Registration Number	F2024408012
Semester	4th Semester — Spring 2025
Submission Date	June 04, 2025

Table of Contents

1. Project Overview

The Cybersecurity Incident Reporting and Tracking System is a web-based platform designed to address the growing challenges organizations face in managing and responding to cybersecurity incidents. As digital infrastructure continues to expand, organizations of all sizes are increasingly exposed to a wide range of security threats, including phishing attacks, malware infections, unauthorized access attempts, ransomware attacks, data breaches, and network intrusions.

Currently, many organizations rely on manual, ad-hoc methods for recording and tracking security incidents, such as email chains, spreadsheets, or informal verbal communication. These approaches result in poor incident tracking, delayed response times, inconsistent documentation, and a lack of centralized visibility into the organization's security posture.

The proposed system provides a centralized, web-accessible platform where users can formally report security incidents, administrators can manage and update incident records, and stakeholders can monitor the status and history of ongoing and resolved incidents. The system will support evidence attachment, incident categorization, severity classification, and dashboard statistics to empower security teams to respond faster and more effectively.

By transitioning from manual processes to a structured digital platform, organizations can significantly improve their incident response efficiency, maintain comprehensive audit trails, and ensure that all security events are properly documented and addressed.

2. Problem Statement

The global cybersecurity landscape is evolving rapidly, and organizations face unprecedented levels of digital threats. Despite the availability of sophisticated security tools, many organizations still lack structured processes for reporting and managing cybersecurity incidents. The following problems have been identified:

2.1 Increasing Cybersecurity Threats

Organizations face a growing number of sophisticated cyber threats on a daily basis. The frequency and complexity of attacks such as ransomware, phishing campaigns, and unauthorized access attempts continue to rise. Without a dedicated tracking system, security teams struggle to maintain awareness of all active incidents simultaneously.

2.2 Lack of Centralized Incident Management

Most small to medium organizations do not possess a centralized system for recording security incidents. Incidents are reported through emails, chat messages, or verbal communication, making it difficult to maintain consistent records or analyze patterns across multiple incidents.

2.3 Difficulty in Monitoring Incident Status

Without a tracking platform, it becomes nearly impossible to monitor the lifecycle of a security incident from initial report to full resolution. Status updates are often lost or delayed, causing confusion among team members about the current state of an incident.

2.4 Inefficient Communication

Incident response requires coordination between multiple stakeholders including reporters, security analysts, and IT administrators. Fragmented communication channels lead to duplication of effort, missed updates, and slower response times.

2.5 Inadequate Documentation and Reporting

Proper documentation of security incidents is essential for regulatory compliance, post-incident analysis, and improvement of security policies. In the absence of a structured system, documentation is often incomplete, inconsistent, or inaccessible when needed.

3. Project Objectives

The primary objectives of the Cybersecurity Incident Reporting and Tracking System are as follows:

1. Develop a fully functional, web-based cybersecurity incident management platform accessible from any modern browser.
2. Enable registered users to formally report security incidents with structured data including title, category, severity, and description.
3. Allow administrators to review, update, assign, and resolve incidents through a dedicated management interface.
4. Store all incident information securely in a PostgreSQL relational database hosted on Supabase.
5. Provide search and filtering functionality to allow users to locate specific incidents quickly based on category, severity, or status.
6. Generate real-time dashboard statistics presenting an overview of total, open, in-progress, and resolved incidents.
7. Support evidence attachment to incidents to allow file uploads relevant to reported security events.
8. Deploy the complete application online via Vercel (frontend) and Render (backend) for remote accessibility.
9. Implement secure user authentication and role-based access control using JWT tokens.

4. Scope of the Project

4.1 In Scope

The following features and components are within the scope of this project:

- User registration with email and password, and role-based access (user / admin).
- Secure user login and logout with JWT-based session management.
- Incident reporting form with fields for title, category, severity, and detailed description.
- Full incident management capabilities for administrators including create, read, update, and delete operations.
- Incident status tracking through defined lifecycle states: Open, In-Progress, and Resolved.
- Search incidents by keyword and filter incidents by category, severity, and status.
- Dashboard analytics page showing incident counts and status distribution.
- Evidence file upload feature linked to specific incidents.
- PostgreSQL database storage with structured relational schema.
- Cloud deployment: Vercel (frontend), Render (backend), Supabase (database).

4.2 Out of Scope

The following items are explicitly outside the scope of this project:

- Real-time automated threat detection or intrusion detection systems.
- AI-based attack prediction or machine learning models.
- Integration with network monitoring infrastructure or live network traffic analysis.
- Security Information and Event Management (SIEM) platform integration.
- Mobile application development (Android or iOS).
- Automated email or SMS notification systems.

5. Target Users

The system is designed to serve a range of user types within organizational and educational contexts:

User Type	Role & Responsibilities
Security Analysts	Primary power users who monitor all reported incidents, investigate security events, update incident statuses, and generate reports for management review.
IT Administrators	System administrators responsible for managing the platform, overseeing user accounts, and ensuring that incidents are escalated to the appropriate teams.
Employees	End users within an organization who can report suspicious activity or security incidents they encounter during day-to-day operations.
Students	Computer science and cybersecurity students at universities who use the platform as a learning tool for incident management concepts and practical lab exercises.
Small Organizations	Businesses or teams without dedicated security infrastructure who require an accessible and affordable incident tracking system to meet basic security management needs.

6. Main Features

The following table outlines the core features of the Cybersecurity Incident Reporting and Tracking System:

Feature	Description
User Registration	Allows new users to create an account using a full name, email address, and password. Validates input and hashes passwords before database storage.
User Login	Authenticates registered users using email and password credentials. Returns a JWT token for session management upon successful authentication.
JWT Authentication	Secures all protected API endpoints using JSON Web Token authentication. Tokens carry user identity and role for server-side authorization.
Incident Reporting	Provides a structured form enabling users to submit cybersecurity incidents including title, type/category, severity level, and a detailed description.
Incident Management	Allows administrators to view all reported incidents and perform create, read, update, and delete (CRUD) operations on incident records.

Incident Status Tracking	Tracks the lifecycle of each incident through three states: Open (newly reported), In-Progress (under investigation), and Resolved (fully addressed).
Search Incidents	Provides a keyword-based search across incident titles and descriptions, enabling rapid retrieval of specific incident records.
Filter Incidents	Enables users to filter the incident list by category, severity level, or current status to narrow results according to specific investigation criteria.
Dashboard Statistics	Displays a real-time summary view with counts of total incidents, open incidents, in-progress incidents, and resolved incidents for quick situational awareness.
User Profile Management	Allows users to view and update their personal profile information including full name and password.
Evidence Upload	Permits users to attach files or screenshots relevant to an incident, with uploaded evidence stored securely and linked to the associated incident record.
Incident History Tracking	Maintains an activity log recording all actions taken on an incident, including status changes and updates, providing a full audit trail.

7. Functional Requirements

The following functional requirements define the specific behaviors and capabilities the system must fulfill:

Req. ID	Requirement Name	Description
FR-01	User Registration	The system shall allow new users to register an account by providing a unique email address, full name, password, and role designation. The password must be hashed before storage.
FR-02	User Login	The system shall authenticate users by validating their email and password against stored credentials. Upon success, a JWT access token shall be issued with a configurable expiration period.
FR-03	Create Incident	Authenticated users shall be able to submit a new incident report by filling in a title, incident category, severity level (Low/Medium/High/Critical), and detailed description.
FR-04	View Incidents	All authenticated users shall be able to retrieve and view a list of all incidents. Administrators shall see all incidents; standard users shall see their own reported incidents.
FR-05	Update Incident	Administrators shall be able to update any incident's details including title, description, category, severity, and status. Users may update incidents they have submitted.
FR-06	Delete Incident	Administrators shall be permitted to permanently delete incident records from the database. Deletion shall also remove all associated evidence and activity log entries.
FR-07	Search Incident	The system shall provide a keyword-based search feature that queries incident titles and descriptions and returns matching results.

FR-08	Filter Incident	The system shall allow users to filter the incident list by one or more of the following criteria: incident category, severity level, or current status.
FR-09	Dashboard Statistics	The system shall display aggregated statistics including total incident count and the count of incidents grouped by status, rendered as a visual dashboard.
FR-10	Upload Evidence	Users shall be able to attach supporting files to an incident report. Accepted file types include images and documents (PDF, PNG, JPG). Files are stored on the server and linked by incident ID.
FR-11	Manage Users	Administrators shall have access to a user management interface to view all registered users, update user roles, and deactivate or delete user accounts.
FR-12	View Incident Details	Any authenticated user shall be able to open a dedicated detail view for a specific incident, displaying all fields, uploaded evidence files, and the complete activity log.

8. Non-Functional Requirements

Category	Requirement
Security	All API endpoints must be protected with JWT authentication. Passwords must be hashed using bcrypt. HTTPS must be enforced in production. All database queries must use parameterized statements to prevent SQL injection.
Performance	The system should return API responses within 500 milliseconds under normal load conditions. Dashboard statistics queries should be optimized using database indexing.
Reliability	The system should maintain a minimum uptime of 99% in production. The database must enforce ACID compliance to prevent data corruption during concurrent operations.
Scalability	The application architecture should support horizontal scaling. The database design should accommodate growth in incident records without degradation in query performance.
Availability	The system must be accessible 24 hours a day, 7 days a week via the public internet from any modern web browser without requiring software installation.
Maintainability	All code must follow consistent naming conventions, be well-commented, and organized into logical modules. A README file shall document setup and deployment procedures.
Usability	The user interface must be intuitive, responsive, and accessible on both desktop and mobile screen sizes. Error messages must be descriptive and helpful to users.

9. Frontend Design (some are optional)

The frontend consists of eight core pages, each designed to deliver a distinct user experience. All pages will be built with HTML5 and CSS3, employing a consistent design language featuring the project color palette, responsive layout, and clear typography.

Page 1: Home Page

Purpose: Serve as the public-facing landing page for the system, introducing the platform to visitors.

Contents: A hero section with the system title and a call-to-action button directing users to register or log in. A feature highlights section presenting key capabilities such as incident tracking, dashboard analytics, and secure

reporting. A brief section explaining the problem the system addresses. Navigation links to the About, Register, and Login pages.

Page 2: About Page

Purpose: Provide contextual information about the system, its purpose, and its development background.

Contents: A description of the project objectives and motivation. Details of the technologies used in the stack. Information about the developer or team. A section outlining the academic context (OSSD course, UMT).

Page 3: Register Page

Purpose: Allow new users to create a system account.

Contents: A registration form with input fields for full name, email address, password, confirm password, and all new users are assigned the default role of user. Form validation with descriptive error messages for invalid or duplicate entries. A redirect link to the login page for existing users. A submit button that triggers the POST /register API endpoint.

Page 4: Login Page

Purpose: Authenticate existing users and initiate a session.

Contents: A login form with fields for email address and password. Client-side validation and descriptive error handling for failed authentication. A 'Remember Me' checkbox option. A link to the registration page for new users. On successful authentication, the user is redirected to the Dashboard page.

Page 5: Dashboard Page

Purpose: Provide an authenticated user with an at-a-glance overview of all incidents in the system.

Contents: Statistics cards displaying the total number of incidents, open incidents, in-progress incidents, and resolved incidents. A recent incidents table showing the latest five submissions. Quick-access navigation buttons to Report Incident and Manage Incidents pages. A welcome banner displaying the logged-in user's name.

Page 6: Report Incident Page

Purpose: Allow authenticated users to formally submit a new cybersecurity incident.

Contents: A structured incident reporting form with fields for incident title, category (dropdown: Phishing, Malware, Unauthorized Access, Ransomware, Data Breach, Network Intrusion, Other), severity level (dropdown: Low, Medium, High, Critical), and detailed description (textarea). An evidence file upload control. A submit button that calls the POST /incidents API endpoint. Confirmation message displayed upon successful submission.

Page 7: Manage Incidents Page

Purpose: Provide administrators with a full list of incidents and management controls.

Contents: A searchable, filterable data table listing all incidents with columns for ID, title, category, severity, status, reported by, and date. Filter dropdowns for category, severity, and status. An action column with Edit, View Details, and Delete buttons per row. Pagination controls for large incident lists.

Page 8: Incident Details Page

Purpose: Display the complete information for a single incident in a structured read view.

Contents: Full incident metadata including all submitted fields and current status. A status update control visible to administrators for changing the incident lifecycle state. An evidence files section listing all uploaded attachments with download links. An activity log timeline showing all recorded actions taken on the incident with timestamps.

10. Backend API Design

The backend is built on FastAPI (Python) and exposes a RESTful API. All protected endpoints require a valid JWT Bearer token in the Authorization header. The following table defines the complete API surface:

Method	Endpoint	Auth	Description
POST	/register	Public	Register a new user account. Accepts full name, email, password, and role. Returns the created user object (without password hash).
POST	/login	Public	Authenticate a user with email and password. Returns a JWT access token on success.
GET	/users	Admin	Retrieve a list of all registered users. Accessible by administrators only.
GET	/users/{id}	Auth	Retrieve the profile details of a specific user identified by their user ID.
PUT	/users/{id}	Auth	Update the profile information (name, password) of a specific user. Users may only update their own profile; admins may update any.
DELETE	/users/{id}	Admin	Permanently delete a user account by user ID. Restricted to administrators.
POST	/incidents	Auth	Create a new incident report. Accepts title, category, severity, and description. Sets status to 'Open' and records the reporter ID.
GET	/incidents	Auth	Retrieve all incidents. Administrators see all records; standard users see only their own submissions.
GET	/incidents/{id}	Auth	Retrieve the full details of a single incident including all fields, evidence files, and activity log.
PUT	/incidents/{id}	Auth	Update incident fields such as title, description, severity, category, or status. Status changes are logged in the activity log.
DELETE	/incidents/{id}	Admin	Delete an incident record along with all associated evidence and activity log entries.
GET	/incidents/search	Auth	Search incidents by keyword. Queries the title and description fields and returns all matching incidents.
GET	/incidents/filter	Auth	Filter incidents by one or more query parameters: category, severity, and/or status.
GET	/dashboard/stats	Auth	Return aggregated statistics: total incident count and counts grouped by status (Open, In-Progress, Resolved).
POST	/evidence/upload	Auth	Upload a file as evidence for a specific incident. Accepts multipart form data. Stores the file and creates an evidence record linked to the incident ID.

11. Database Selection

11.1 Selected Database: PostgreSQL

PostgreSQL has been selected as the primary relational database management system for this project. The decision is based on the following justifications:

- Reliability: PostgreSQL is a production-grade, enterprise-class open-source RDBMS with over 35 years of active development, widely recognized for its stability and correctness.
- Security: PostgreSQL supports row-level security, role-based access control, SSL connections, and column-level encryption — features essential for a cybersecurity-focused platform.
- ACID Compliance: All transactions in PostgreSQL fully satisfy Atomicity, Consistency, Isolation, and Durability properties, ensuring data integrity even in the event of system failures.
- Structured Relational Design: The incident management domain involves well-defined relational entities (users, incidents, evidence, logs) that map naturally to a normalized relational schema.
- FastAPI Integration: The SQLAlchemy ORM provides excellent, first-class support for PostgreSQL within FastAPI, enabling clean and maintainable data access layers.
- Supabase Compatibility: Supabase provides a fully managed PostgreSQL hosting service with a generous free tier, built-in REST API, and real-time capabilities, making it ideal for academic deployment.

12. Database Design

Table 1: Users

Field	Data Type	Constraints	Description
user_id	UUID / SERIAL	PRIMARY KEY, NOT NULL	Unique identifier for each user record.
full_name	VARCHAR(150)	NOT NULL	The full name of the registered user.
email	VARCHAR(255)	UNIQUE, NOT NULL	User email address, used as the login credential.
password_hash	TEXT	NOT NULL	Bcrypt-hashed password string. Never stored in plain text.
role	VARCHAR(20)	DEFAULT 'user'	User role designation: 'user' or 'admin'. Controls access permissions.
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp of account creation, automatically set on record insert.

Table 2: Incidents

Field	Data Type	Constraints	Description
incident_id	UUID / SERIAL	PRIMARY KEY, NOT NULL	Unique identifier for each incident record.
title	VARCHAR(255)	NOT NULL	Short descriptive title summarizing the incident.
description	TEXT	NOT NULL	Full narrative description of the incident including observed symptoms and impact.
category	VARCHAR(100)	NOT NULL	Incident classification type: Phishing, Malware, Unauthorized Access, etc.
severity	VARCHAR(20)	NOT NULL	Severity level: Low, Medium, High, or Critical.

status	VARCHAR(20)	DEFAULT 'Open'	Current lifecycle state: Open, In-Progress, or Resolved.
reported_by	INTEGER / UUID	FOREIGN KEY → Users	References the user_id of the user who submitted the incident.
created_at	TIMESTAMP	DEFAULT NOW()	Timestamp when the incident was first reported.
updated_at	TIMESTAMP	DEFAULT NOW()	Timestamp automatically updated whenever the incident record is modified.

Table 3: Evidence

Field	Data Type	Constraints	Description
evidence_id	UUID / SERIAL	PRIMARY KEY, NOT NULL	Unique identifier for each evidence file record.
incident_id	INTEGER / UUID	FOREIGN KEY → Incidents	References the incident to which this evidence belongs.
file_name	VARCHAR(255)	NOT NULL	Original filename of the uploaded evidence file as provided by the user.
file_path	TEXT	NOT NULL	Server-side storage path or cloud URL where the uploaded file is persisted.
uploaded_at	TIMESTAMP	DEFAULT NOW()	Timestamp recording when the evidence file was uploaded to the system.

Table 4: Activity Logs

Field	Data Type	Constraints	Description
log_id	UUID / SERIAL	PRIMARY KEY, NOT NULL	Unique identifier for each activity log entry.
incident_id	INTEGER / UUID	FOREIGN KEY → Incidents	References the incident on which the recorded action was performed.
action	TEXT	NOT NULL	Description of the action performed, e.g., 'Status changed to In-Progress'.
performed_by	INTEGER / UUID	FOREIGN KEY → Users	References the user_id of the user who performed the action.
timestamp	TIMESTAMP	DEFAULT NOW()	Exact date and time at which the action was recorded in the system.

13. Entity Relationship Description

The database schema follows a normalized relational design. The following entity relationships are defined:

Relationship	Cardinality	Description

Users → Incidents	One-to-Many	A single registered user can create and submit multiple incident reports. Each incident record maintains a foreign key (reported_by) that references exactly one user.
Incidents → Evidence	One-to-Many	A single incident can have multiple evidence files attached to it. Each evidence record holds a foreign key (incident_id) linking it to its parent incident.
Incidents → Activity Logs	One-to-Many	A single incident can accumulate many activity log entries over its lifecycle. Each log entry records one action and references the incident and the user who performed it.
Users → Activity Logs	One-to-Many	Each activity log entry records the user who performed the action via the performed_by foreign key, linking back to the Users table.

Textual ERD Summary:

Users (user_id PK) ———< Incidents (incident_id PK, reported_by FK)

Incidents (incident_id PK) ———< Evidence (evidence_id PK, incident_id FK)

Incidents (incident_id PK) ———< Activity_Logs (log_id PK, incident_id FK)

Users (user_id PK) ———< Activity_Logs (log_id PK, performed_by FK)

14. Technology Stack

Component	Technology	Justification
Frontend	HTML5	Provides semantic, standards-compliant markup for building accessible and well-structured web pages without additional framework overhead.
Styling	CSS3	Enables custom, responsive design using flexbox, grid, and media queries without the dependency of a CSS framework.
Backend Framework	FastAPI (Python)	FastAPI offers high performance, automatic OpenAPI documentation, native async support, and excellent integration with SQLAlchemy and JWT.
Programming Language	Python 3.11+	Python is the dominant language in cybersecurity tooling and data processing, and aligns with the student's existing proficiency.
Database	PostgreSQL	A robust, ACID-compliant relational DBMS ideally suited to structured incident data with well-defined relationships.
ORM	SQLAlchemy	Provides a Pythonic abstraction over database queries, reduces raw SQL exposure, and simplifies schema migrations.
Authentication	JWT (JSON Web Tokens)	Stateless token-based authentication enables secure, scalable API access control without server-side session storage.
Database Hosting	Supabase	Managed PostgreSQL with a free tier, web-based dashboard, and built-in REST API generation, suitable for academic projects.
Backend Deployment	Render	Render provides free-tier Python web service hosting with automatic GitHub deployment pipelines and environment variable management.

Frontend Deployment	Vercel	Industry-standard static hosting with global CDN, automatic HTTPS, and seamless deployment from GitHub repositories.
Version Control	GitHub	Provides distributed version control, issue tracking, and project management through a widely-used open-source platform.

15. System Architecture

The system follows a three-tier client-server architecture separating presentation, logic, and data concerns:

Tier	Component	Technology	Responsibility
Presentation	Frontend	HTML5 / CSS3	Renders the user interface in the browser. Sends HTTP requests to the backend API and displays responses to the user.
Logic	Backend API	FastAPI / Python	Processes incoming requests, enforces authentication and authorization, executes business logic, and communicates with the database.
Data	Database	PostgreSQL / Supabase	Persists all application data including user accounts, incident records, evidence references, and activity logs.

Architecture Flow:

1. The end user accesses the application through a web browser by navigating to the Vercel-hosted frontend URL.
2. The browser renders the HTML/CSS interface and captures user interactions such as form submissions and button clicks.
3. JavaScript in the frontend sends HTTP requests (fetch / XMLHttpRequest) to the FastAPI backend hosted on Render, including JWT tokens in the Authorization header.
4. FastAPI validates the incoming request, verifies the JWT token, and processes the business logic (e.g., creating an incident, querying statistics).
5. FastAPI communicates with the PostgreSQL database on Supabase via SQLAlchemy, executing parameterized queries.
6. The database returns results to FastAPI, which serializes the data into a JSON response and sends it back to the browser.
7. The frontend receives the JSON response and dynamically updates the page to reflect the new state.

16. Deployment Plan

16.1 Frontend Deployment — Vercel

10. Develop and test all HTML/CSS files locally.
11. Push the frontend source files to a dedicated GitHub repository.
12. Create a Vercel account and connect it to the GitHub repository.
13. Configure the Vercel project root and build settings for static HTML deployment.
14. Set any environment variables (e.g., backend API base URL) in the Vercel dashboard.
15. Deploy via Vercel's automatic pipeline; every push to the main branch triggers a new deployment.

16.2 Backend Deployment — Render

16. Ensure the FastAPI application includes a requirements.txt file listing all Python dependencies.
17. Push the backend source code to a GitHub repository.
18. Create a Render account and connect the GitHub repository as a new Web Service.
19. Configure the start command: uvicorn main:app --host 0.0.0.0 --port 10000.
20. Add all required environment variables to Render's dashboard including the database connection string and JWT secret key.
21. Deploy the service; Render automatically builds and restarts on each code push.

16.3 Database Hosting — Supabase

22. Create a Supabase project and obtain the PostgreSQL connection string.
23. Define the database schema using SQLAlchemy models or direct SQL in the Supabase SQL Editor.
24. Store the Supabase connection string securely as an environment variable in both the local .env file and the Render dashboard.
25. Monitor database performance through the Supabase project dashboard.

17. Security Considerations

Security Measure	Implementation
Password Hashing	All user passwords are hashed using the bcrypt algorithm before being stored in the database. Plain-text passwords are never persisted or transmitted.
JWT Authentication	Protected endpoints validate a signed JWT token sent in the Authorization header. Tokens expire after a configured period to limit exposure if compromised.
Input Validation	FastAPI's Pydantic models enforce strict type checking and length constraints on all incoming request data. Invalid input is rejected with a 422 Unprocessable Entity response.
SQL Injection Prevention	All database queries are executed via SQLAlchemy's parameterized query interface, ensuring that user-supplied data is never interpolated directly into SQL statements.
HTTPS Enforcement	Both Vercel and Render enforce HTTPS by default in production environments, ensuring all data transmitted between the client and server is encrypted in transit.
Environment Variables	Sensitive configuration values including the database connection string, JWT secret key, and API keys are stored exclusively in environment variables and never hardcoded in source code.

18. Expected Output

Upon successful completion of the project, the following deliverables and capabilities will be available:

- A fully deployed, cloud-accessible web application available at a public Vercel URL.
- Secure user registration and login system with role-based access control for users and administrators.
- A functional incident reporting workflow enabling users to submit structured cybersecurity incident records from any browser.
- A comprehensive incident management interface for administrators to create, view, update, and delete incident records.
- Real-time dashboard statistics presenting an operational overview of all incidents by status.
- Search and filtering capabilities allowing rapid retrieval of specific incidents by category, severity, or status.
- An evidence upload system linking files to their associated incidents.

- A complete activity log for each incident providing a full audit trail of all actions.
- Database-backed persistent storage of all incident, user, evidence, and activity log data on Supabase.
- A well-documented GitHub repository with README, API documentation, and deployment instructions.

19. Future Enhancements

The following enhancements are planned for future development phases beyond the scope of this project:

Enhancement	Description
Email Notifications	Automatic email alerts to relevant stakeholders when a new incident is reported or when an incident status changes, using services such as SendGrid or SMTP.
AI-Based Incident Classification	Integration of a machine learning model to automatically suggest the category and severity of newly submitted incidents based on the description text.
Threat Intelligence Integration	Connecting the system to public threat intelligence feeds (e.g., MISP, AlienVault OTX) to enrich incident records with contextual threat data.
Mobile Application	Development of a React Native or Flutter mobile application providing the same incident reporting and management capabilities on Android and iOS devices.
Real-Time Monitoring	Implementation of WebSocket-based live updates to push new incident notifications to the dashboard without requiring a page refresh.
Advanced Analytics	Enhanced reporting capabilities including trend analysis charts, incident frequency over time, mean time to resolution (MTTR) metrics, and exportable PDF reports.

20. Conclusion

The Cybersecurity Incident Reporting and Tracking System addresses a genuine and critical gap in how organizations manage cybersecurity events. By providing a centralized, web-accessible platform for reporting, tracking, and resolving security incidents, the system will significantly improve incident response efficiency, enhance documentation quality, and support better-informed security decision-making.

The project is grounded in a carefully selected and justified technology stack — FastAPI, PostgreSQL, and modern cloud deployment platforms — that reflects current industry practices and provides a solid foundation for future enhancement. The comprehensive relational database design, RESTful API architecture, and role-based security model together form a system that is not only functional for academic submission but also representative of real-world security operations tooling.

Through the development of this system, the project will demonstrate practical competency in full-stack web development, database design, API engineering, cloud deployment, and cybersecurity-oriented software design — meeting all the objectives set forth for the OSSD Final Semester Project, Phase 1.

— End of Phase 1 Project Proposal Document —